Literature Survey for Network Intrusion Detection with Concept Drift Handling on UNSW-NB15 dataset

1. Scope of survey

This literature survey splits the main project topic into three major areas: machine learning based network intrusion detection (NIDS), concept drift detection and adaptation, and streaming learning applied to network security scenarios. 21 recent research papers were reviewed.

2. Thematic Review

2.1 Machine Learning NIDS and the choice of dataset

Ahmad et al. [1] surveyed ML and deep-learning NIDS and found that most work reports high accuracy on a fixed train/test split, with little attention to how models behave when traffic changes. Sarhan et al. [2] used the UNSW-NB15 dataset along with other standard datasets like NetFlow, and made a cross-dataset comparison. It can be concluded that UNSW-NB15 is one of the more preferable datasets for developing a NIDS, as it more accurately reflects modern network protocols than several other outdated ones. Layeghy et al. [3] compared synthetic and real-world NIDS datasets and argued that benchmark results on older synthetic data can overstate real-world performance. Engelen et al. [4] showed that labelling and flow-construction errors in CICIDS2017 had inflated published results, a warning that dataset quality matters as much as the algorithm. Yang et al. [5] proposed MTH-IDS, a multi-tier hybrid IDS using tree-based learners and optimisation, which reaches very high detection rates on standard benchmarks, again under static evaluation. Grinsztajn et al. [6] showed that tree-based models still outperform deep networks on typical tabular data. This supports our choice of XGBoost as the main classifier for UNSW-NB15's tabular flow features.

2.2 Concept drift

Agrahari and Singh [7] reviewed drift detection in data streams and grouped detectors into error-rate based (ADWIN), distribution-based (KS-test), and ensemble approaches. Bayram et al. [8] observed that performance-aware detectors need truth labels and may react late as they only see drift in the data once accuracy has already dropped. Hinder et al. [9] surveyed drift monitoring and separated drift in the input distribution from changes in the label relationship and noted that many detectors cannot differentiate between the two types of drift. Lukats et al. [10] benchmarked unsupervised drift detectors on real data streams and noted that results may vary drastically depending on the datasets and preprocessing, with extremely high false-alarm rates for some methods. Together, these works [7–10] motivate comparing a label-based detector (ADWIN) with a label-free one (KS test) on the same stream.

2.3 Streaming learning applied to network security

Andresini et al. [11] introduced INSOMNIA, which handles drift in NIDS through semisupervised active learning and reports better robustness than static models on CIC-based data. Yang et al. [12] proposed CADE, which detects drift by contrasting known data and new data. Barbero et al. [13] showcased drift in malware classification and observed that popular drift-detection methods are sensitive to evaluation settings. Chen et al. [14] studied continuous learning for Android malware detection and found that how new labelled data is chosen strongly affects long-term accuracy. Malekghaini et al. [15] studied data drift in encrypted traffic classification and found significant performance loss over time that retraining only partly recovers. Yang and Shami [16] proposed OWAD, a lightweight drift detection and adaptation framework for IoT data streams that retrains when drift is signalled. These papers [11–16] show that drift in security data is real and costly, but most use malware, encrypted traffic, or IoT data. Only few study UNSW-NB15 flows with different drift types.

Montiel et al. [17] released River, a Python library for streaming ML that includes Hoeffding trees and the Adaptive Random Forest (ARF), which replaces degraded trees using per-tree ADWIN detectors. Bahri et al. [18] reviewed data-stream analysis tools and tasks and highlighted prequential (test-then-train) evaluation as the standard protocol. Gunasekara et al. [19] surveyed online streaming continual learning and discussed the trade-off between adapting quickly and forgetting useful old knowledge. These papers [17-19] show that online learners usually avoid explicit retraining and are usually compared on generic benchmark streams, not against a retrain-on-alarm strategy.

2.4 Unknown and zero-day attacks

Zoppi et al. [20] evaluated unsupervised algorithms for zero-day detection and concluded that no single algorithm is reliable across attack types. Verkerken et al. [21] studied model generalisation for intrusion detection with unsupervised methods and found weaker generalisation to unseen attack classes than to seen ones. This matters for our abrupt drift scenario, in which attack families unseen in warm-up appear. It also explains why a purely unsupervised classifier is expected to lose recall on them [20, 21]

3. Comparison

S. No	Reference	Data	Main Finding	Limitation relevant to this project
1.	Ahmad et al. [1]	Multiple NIDS sets	Strong static accuracy	No drift evaluation
2.	Sarhan et al.	UNSW-NB15	Enables	Static evaluation

	[2]	and others (NetFlow)	cross-dataset tests	
3.	Layeghy et al. [3]	Synthetic vs real NIDS sets	Synthetic benchmarks overstate performance	No streaming evaluation
4.	Engelen et al. [4]	CICIDS2017	Labelling errors inflate results	Warns about data quality, no adaptation
5.	Yang et al. [5]	IoV/CICIDS	Very high detection rates	No drift study
6.	Grinsztajn et al. [6]	Tabular benchmarks	Trees stay strong on tabular data	Not security specific
7.	Agrahari and Singh [7]	Review	Taxonomy of detectors	No empirical comparison
8.	Bayram et al. [8]	Review	Detection can lag degradation	Label-based only
9.	Hinder et al. [9]	Review	Drift types need separate handling	Survey only
10.	Lukats et al. [10]	Data streams	Results vary, false alarms are common	Not NIDS specific
11.	Andresini et al. [11]	NIDS data	Better than static models	Needs labelling, different data
12.	Yang et al. [12]	Malware intrusion	Detects and explains drift	No retraining strategy
13.	Barbero et al. [13]	Malware data	Results depend on evaluation setting	Malware domain
14.	Chen et al. [14]	Android malware data	Sample selection matters	Malware domain
15.	Malekghaini et al. [15]	Encrypted traffic	Large performance loss over time	Encrypted traffic, not flow attacks
16.	Yang and Shami [16]	IoT streams	Lightweight retraining works	One detector; IoT data

17.	Montiel et al. [17]	Streaming	Practical streaming toolkit	Generic benchmarks
18.	Bahri et al. [18]	Review	Prequential protocol is standard	Survey only
19.	Gunasekara et al. [19]	Review	Open tradeoffs	Survey only
20.	Zoppi et al. [20]	Several datasets	No algorithm is universally reliable	No streaming scenario
21.	Verkerken et al. [21]	Intrusion data	Weaker on unseen attack classes	No adaptation strategy

4. Analysis

Evaluation protocol: The NIDS studies in 2.1 [1, 2, 5] mostly report accuracy or F1 from one train/test split. Such numbers say nothing about how long a model stays valid. The dataset-quality studies [3, 4] add that even those static numbers can be inflated.

Detector choice: Performance-based detectors such as ADWIN need labels and fire only after errors rise [8]. Distribution-based detectors need no labels and can fire earlier, but can raise false alarms, particularly when many features are tested [10]. The literature [7, 8, 10] describes both methods but rarely compares them on one controlled NIDS stream with measured detection delay and false alarms

Drift type: Hinder et al. [9] stress that covariate shift and real concept change need different responses. Security drift papers [13–15] mostly use a single natural drift (malware evolution or time passing) and do not separate a new-attack-family drift from a traffic-volume drift in the same experiment.

Adaptation strategy: Retrain-on-alarm approaches [16] and online learners [17] are usually studied separately. Their cost and accuracy are seldom compared against a no-adaptation control under identical conditions.

5. Research Gap

1. G1: Few studies measure how fast a NIDS degrades on UNSW-NB15-style flow data under controlled, separately simulated drift types (new attack families and volume shift).
2. G2: Label-based (ADWIN) and label-free (KS-test) detectors are rarely compared on the same NIDS stream using detection delay and false alarms.

3. G3: Drift-triggered retraining and online learning (ARF) are rarely compared against a static control with per-phase and per-attack-family metrics.

6. Problem statement and Objectives

Problem statement

A supervised NIDS trained on past traffic loses accuracy when new attack types appear or traffic characteristics shift. Operators need to know when this happens, how much it costs, and which adaptation strategy recovers performance at an acceptable retraining cost.

Research questions

RQ1: How much does the accuracy and F1 of a static XGBoost NIDS fall under abrupt (new attack families) and progressive (traffic volume) drift?
RQ2: Which detector, ADWIN or per-feature KS-test, alarms earlier and with fewer false alarms?
RQ3: Does drift-triggered retraining or online ARF recover performance better than a static model?

Objectives

S.No	Objective	Metric of evaluation
O1	Train and evaluate a baseline NIDS on the official UNSW-NB15 train/test files	Report precision, recall, and F1 for XGBoost and Random Forest
O2	Build a 60-window stream (1,000 rows per window, 40% attacks) with abrupt drift at window 20 and progressive volume drift over windows 40–50	Per-phase mean F1 for before, new family, volume-drift, and after phases
O3	Compare ADWIN and KS-test detectors	Detection delay in windows and number of false alarms for each detector
O4	Compare static, drift-triggered retraining and online ARF	Mean F1 per phase, recall per attack family after window 20, and number of retrains

7. Proposed Methodology

- **Data and preprocessing:** UNSW-NB15 (about 257,000 flows, 42 features). A scikitlearn pipeline (log1p and scaling for skewed volume features, scaling for other numerics, one-hot for categoricals) is fitted on training data only and reused everywhere.
- **Models:** XGBoost as the main classifier, Random Forest as a baseline, and River's Adaptive Random Forest as the online learner.
- **Stream design:** a 30,000-row warm-up with normal traffic and old-family attacks, followed by 60 simulated windows, with the two drift types described above.
- **Detection and handling:** ADWIN on the error stream and KS-tests against a 5,000-row reference. Three strategies are compared: static, drift triggered retraining (six-window buffer, cooldown) and online ARF.
- **Evaluation:** prequential, with accuracy, precision, recall, F1, and ROC-AUC. Extra analyses cover per-family recall, detector delay and false alarms, and retrain counts.
- **Tools and feasibility:** Google Colab for training, Python libraries (pandas, scikit-learn, XGBoost, River, SciPy), and a local Streamlit dashboard. All components are open source and run on a standard laptop or free Colab tier, so the plan is feasible within a semester.
- **Known limitations:** drift is simulated, labels are assumed to arrive immediately, parameters are not tuned, and results come from a single seeded run.

8. References

[1] Ahmad, Z., Shahid Khan, A., Wai Shiang, C., Abdullah, J. and Ahmad, F., 2021. Network intrusion detection system: A systematic study of machine learning and deep learning approaches. Transactions on Emerging Telecommunications Technologies, 32(1), p.e4150.

[2] Sarhan, M., Layeghy, S. and Portmann, M., 2022. Towards a standard feature set for network intrusion detection system datasets. Mobile networks and applications, 27(1), pp.357-370. Springer

[3] Layeghy, S., Gallagher, M. and Portmann, M., 2024. Benchmarking the benchmark—Comparing synthetic and real-world Network IDS datasets. Journal of information security and applications, 80, p.103689. Science Direct

[4] Engelen, G., Rimmer, V. and Joosen, W., 2021, May. Troubleshooting an intrusion detection dataset: the CICIDS2017 case study. In 2021 IEEE Security and Privacy Workshops (SPW) (pp. 7-12). IEEE.

[5] Yang, L., Moubayed, A. and Shami, A., 2021. MTH-IDS: A multitiered hybrid intrusion detection system for Internet of Vehicles. IEEE internet of things journal, 9(1), pp.616-632. IEEE.

[6] Grinsztajn, L., Oyallon, E. and Varoquaux, G., 2022. Why do tree-based models still outperform deep learning on typical tabular data?. Advances in neural information processing systems, 35, pp.507-520. NeurIPS Proceedings

[7] Agrahari, S. and Singh, A.K., 2022. Concept drift detection in data stream mining: A literature review. Journal of King Saud University-Computer and Information Sciences, 34(10), pp.9523-9540. Science Direct

[8] Bayram, F., Ahmed, B.S. and Kassler, A., 2022. From concept drift to model degradation: An overview on performance-aware drift detectors. Knowledge-Based Systems, 245, p.108632. Science Direct

[9] Hinder, F., Vaquet, V. and Hammer, B., 2024. One or two things we know about concept drift—a survey on monitoring in evolving environments. Part A: detecting concept drift. Frontiers in Artificial Intelligence, 7, p.1330257. Frontiers

[10] Lukats, D., Zielinski, O., Hahn, A. and Stahl, F., 2025. A benchmark and survey of fully unsupervised concept drift detectors on real-world data streams. International Journal of Data Science and Analytics, 19(1), pp.1-31. Springer

[11] Andresini, G., Pendlebury, F., Pierazzi, F., Loglisci, C., Appice, A. and Cavallaro, L., 2021, November. Insomnia: Towards concept-drift robustness in network intrusion detection. In Proceedings of the 14th ACM workshop on artificial intelligence and security (pp. 111-122). ACM

[12] Yang, L., Guo, W., Hao, Q., Ciptadi, A., Ahmadzadeh, A., Xing, X. and Wang, G., 2021. {CADE}: Detecting and explaining concept drift samples for security applications. In 30th USENIX Security Symposium (USENIX Security 21) (pp. 2327-2344) USENIX

[13] Barbero, F., Pendlebury, F., Pierazzi, F. and Cavallaro, L., 2022, May. Transcending transcend: Revisiting malware classification in the presence of concept drift. In 2022 IEEE Symposium on Security and Privacy (SP) (pp. 805-823). IEEE.

[14] Chen, Y., Ding, Z. and Wagner, D., 2023. Continuous learning for android malware detection. In 32nd USENIX Security Symposium (USENIX Security 23) (pp. 1127-1144). USENIX

[15] Malekghaini, N., Akbari, E., Salahuddin, M.A., Limam, N., Boutaba, R., Mathieu, B., Moteau, S. and Tuffin, S., 2023. Deep learning for encrypted traffic classification in the face of data drift: An empirical study. Computer Networks, 225, p.109648. Science Direct

[16] Yang, L. and Shami, A., 2021. A lightweight concept drift detection and adaptation framework for IoT data streams. IEEE Internet of Things Magazine, 4(2), pp.96-101. IEEE

[17] Montiel, J., Halford, M., Mastelini, S.M., Bolmier, G., Sourty, R., Vaysse, R., Zouitine, A., Gomes, H.M., Read, J., Abdessalem, T. and Bifet, A., 2021. River: machine learning for streaming data in python. Journal of Machine Learning Research, 22(110), pp.1-8.

[18] Bahri, M., Bifet, A., Gama, J., Gomes, H.M. and Maniu, S., 2021. Data stream analysis: Foundations, major tasks and tools. Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery, 11(3), p.e1405. Wiley

[19] Gunasekara, N., Pfahringer, B., Gomes, H.M. and Bifet, A., 2023, August. Survey on Online Streaming Continual Learning. In IJCAI (pp. 6628-6637).

[20] Zoppi, T., Ceccarelli, A. and Bondavalli, A., 2021. Unsupervised algorithms to detect zero-day attacks: Strategy and application. Ieee Access, 9, pp.90603-90615. IEEE

[21] Verkerken, M., D'hooge, L., Wauters, T., Volckaert, B. and De Turck, F., 2022. Towards model generalization for intrusion detection: Unsupervised machine learning techniques. Journal of Network and Systems Management, 30(1), p.12. Springer